

Expliquer le problème: Exemple pédagogique:

Je veux simuler un vecteur de dimension D tel que : on pose $X = (X^1, \dots, X^D)$, $X^i \in \mathbb{N}^*$:

$$\bullet P(X^1 = x^1, \dots, X^D = x^D) = K x^1.$$

$$\bullet \sum X^i \leq N$$

Regardons $D=3$ et $N=5$:

Valeurs possibles X	(1, 1, 1)	(1, 1, 2)	(1, 1, 3)	(1, 2, 1)	(1, 2, 2)	(1, 3, 1)	(2, 1, 1)	(2, 1, 2)
Probabilité	K	K	K	K	K	K	$2K$	$2K$

Valeurs possibles X	(2, 2, 1)	(3, 1, 1)
Probabilités	$2K$	$3K$

X a 10 états possibles

Par que ce soit une probabilité, $K = \frac{1}{15}$.

Par simuler X , c'est simple:

$$K = \frac{1}{15}$$

$$\text{proba} = (K, K, \dots, 2K, \dots, 3K)$$

$$\text{sample} (1:10, \text{prob} = \text{proba}).$$

Regardons maintenant avec $D=7$ et $N=31$.

↳ impossible à main nu.

Si D et N deviennent grands, le nombre de cas va augmenter de manière exponentielle, et le temps de calcul de K par un ordinateur va devenir très long ($\rightarrow \infty$).

Détermination de K : Problème NP -difficile (ou NP -Hard, ou NP complet).

Institut Clay: 1 000 000 \$ par résoudre 1 des 7 problèmes:

- Conjecture Poincaré (Résolu Perelman)
- Conjecture $P=NP$.

Peut-on toujours trouver un algorithme en temps polynomial par résoudre des problèmes de taille non polynomiale?

La majorité des experts pensent que $P \neq NP$.

Étape 1: Vers la résolution du problème.

Algorithme de Metropolis (-Hastings).

$\Rightarrow$ Nicholas Metropolis (Algorithmes dans les données 1950).

Rapports: Chaîne de Markov: Q = Matrice de transition

(Z_n) : chaîne de Markov

$Q(x, y) = P(Z_{n+1} = y | Z_n = x)$ et $\sum_y Q(x, y) = 1$

* Iréductibilité: Tous les états communiquent

* Probabilité invariante: $\pi(y) = P(Z_1 = y)$

$$\text{probabilité totale} \Rightarrow \sum_x P(Z_1 = y \cap Z_0 = x) = \sum_x P(Z_1 = y | Z_0 = x) \underbrace{P(Z_0 = x)}_{\pi}$$

π est invariante si $\pi Q = \pi$.

Théorème: Ergodique ($\Leftrightarrow$ LG-N): $\frac{1}{n} \sum_{i=1}^n f(Z_i) \xrightarrow[n \rightarrow \infty]{p.s.} \int f(z) d\pi(z) = E_{\pi}[Z_0]$.

C'est le théorème qui permet d'envisager de faire du Monte-Carlo.

Algorithme de Metropolis

But: Simuler un vecteur $X \in E^D$ qui suit une loi de probabilité π .

E est un espace d'état fini.

Dans notre exemple, $E = \{1, \dots, N-D+1\}$

Principe: Construire une chaîne X_n qui admet π comme probabilité invariante.

Étape 1: Initialisation: On choisit x_0 .

↳ par exemple: $x_0 = (1, 1, 1)$.

Étape 2: Transition: On va introduire une chaîne de Markov, dite "instrumentale", de probabilité de transition q .

Remarque: Il faut savoir la simuler!

Exemple: $(1, 2, 1)$

Je tire une coordonnée au hasard, puis transforme cette coordonnée avec une loi discrète uniforme sur les valeurs possibles

Étape 3: Acceptation ou refus de la transition:

On calcule $\alpha(X_n, Y_n) = \min\left(\frac{\pi(Y_n) q(X_n, Y_n)}{\pi(X_n) q(Y_n, X_n)}, 1\right)$.

Remarque: $\pi(Y_n) = \pi((Y_n^1, Y_n^2, Y_n^3))$ et $\pi(X_n) = K X_n^4$.

$$\begin{aligned} &= K Y_n^4 \\ \Rightarrow \frac{\pi(Y_n)}{\pi(X_n)} &= \frac{Y_n^4}{X_n^4} \Rightarrow K \text{ a disparu!!!} \end{aligned}$$

On peut implémenter cet algorithme sans connaître la valeur de la constante qui est impossible à calculer.

(19)

On simule une $U_{[0,1]} \Rightarrow u$.

Si $u \leq \alpha$, on accepte: $X_{n+1} = Y_n$.

Si $u > \alpha$, on rejette: $X_{n+1} = X_n$.

(X_n) est une chaîne de Markov: $y \neq x, P(X_{n+1} = y | X_n = x) = P(Y_n = y \cap U_n \leq \alpha(Y_n, X_n) | X_n = x)$
 $= Q(x, y) P(U_n \leq \alpha(X_n, Y_n)) = Q(x, y) \alpha(X_n, Y_n).$

Proposition: la probabilité invariante de (X_n) est π .

Démonstration: On a $\pi(x) P(X_{n+1} = y | X_n = y) = \pi(x) Q(x, y) \alpha(x, y)$
 $= \min(\pi(y) Q(y, x), \pi(x) Q(x, y)).$

$\pi(x) P(X_{n+1} = y | X_n = x) = \pi(y) P(X_{n+1} = x | Y_n = y).$

↳ la chaîne est "réversible".

On a donc $\sum_x P(X_{n+1} = y | X_n = x) \pi(x) = \pi(y) \sum_x \overbrace{P(X_{n+1} = x | X_n = y)}^{=1}$

↳ π est la mesure invariante de (X_n) .

Intuition: Je suppose que Q symétrique: $Q(x, y) = Q(y, x)$.

$\alpha(X_n, Y_n) = \min(\frac{\pi(Y_n)}{\pi(X_n)}, 1)$

Si $\pi(Y_n) > \pi(X_n) \Rightarrow \alpha = 1$.

On garde l'état Y_n , ce qui est normal puisqu'il est plus vraisemblable par π .

Si $\pi(Y_n) < \pi(X_n)$, on va le garder avec une probabilité proportionnelle au rapport de vraisemblance.

18/11/25

Théorème des 4 couleurs

Il faut au moins 4 couleurs

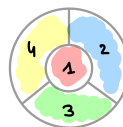
Fin XIX^{ème} siècle: On sait que ça marche avec 5 couleurs

1^{ère} preuve 1975: Toutes les cartes

1^{ère} étape: Il suffit de le montrer sur un nombre fini de cartes (≈ 2000)

2^{ème} étape: Le vérifier par ses 2000 cartes $\Rightarrow$ Ord

1^{ère} preuve: assisté par ordinateur



But: Colorier la carte en utilisant un algorithme stochastique

Modélisation du problème: Je vais définir un vecteur G , de taille N , le nombre de régions à colorier, tel que:

$G[i] = 3 \Rightarrow$ la i ^{ème} région est colorée par la couleur 3.

$G[i] = j$ avec $i \in \{1, \dots, N\}$ et $j \in \{1, 2, 3, 4\}$

Par les départements, taille du problème: $4^{95} = 1.5 \times 10^{57}$! (Nombre de coloriage possible).

Algorithme de base: Faites tous les coloriage possibles et retenir ceux qui marchent et impossible.

18

Algorithme dit de "Brut Forcing", ou "recherche exhaustive".

Etape suivante: Ecrire le problème sous forme d'un problème d'optimisation.

S'introduit la fonction: $H(G) = \sum_{i=1}^{N-1} \sum_{j=i+1}^N \underbrace{\prod_{i,j \text{ voisins}}}_{\text{compte le nombre de conflits}} \prod_{\{G[i] = G[j]\}}$.

Théorème des 4 couleurs: $\min_G H(G) = 0$.

S'ai besoin de la matrice V de voisinage: $V[i,j] = \begin{cases} 1 & \text{si départements voisins} \\ 0 & \text{sinon} \end{cases}$

Introduire de l'aléatoire: "Mesure de Gibbs": $\pi(G) = \frac{1}{Z} e^{-\beta H(G)}$, $\beta > 0$ avec $Z = \sum_{G \text{ possible}} e^{-\beta H(G)}$

Remarque: Z n'est pas calculable

Remarque: $\pi(G)$ est autant plus fort que $H(G)$ est faible

$\Rightarrow$ Si je sais simuler la loi π , si je tire au hasard avec cette loi π , j'ai plus de chance d'obtenir un "bon" coloriage qu'un "mauvais".

$\Rightarrow$ On a l'algorithme de Metropolis pour simuler cette loi.

1^{ère} étape programmer la fonction H , dans le code



Se tire un département au hasard.

Se change sa couleur de façon uniforme

↳ Transition "simple", symétrique.

$G = (2 \ 3 \ 3 \ 1 \ \dots \ 2 \ \dots \ 3 \ 4 \ 1)$

$G_{\text{new}} = (\dots)$